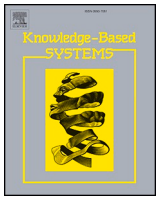
Knowledge-Based Systems 289 (2024) 111474



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





An effective collaboration evolutionary algorithm for multi-robot task allocation and scheduling in a smart farm

Hengwei Guo [a], Zhonghua Miao [a,*], JC Ji [b], Quanke Pan [a]

[a] School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200072, PR China
[b] School of Mechanical and Mechatronic Engineering, University of Technology Sydney, Australia

ARTICLE INFO

Key words:
Smart farms
Multi agriculture robots
Task allocation and scheduling
Artificial bee colony algorithm
Constructive heuristics

ABSTRACT

The rapid development of intelligent and unmanned technologies has created new opportunities and challenges for smart farms. Agricultural production is increasingly incorporating intelligent robots for various farming activities. This paper investigates the task allocation and scheduling for multiple agricultural robots operating in a smart farm. For addressing this problem, a collaborative discrete bee colony algorithm (CDABC) is introduced to provide a viable solution, aiming to optimize the system efficiency in smart farms. To enhance the global search ability, several solution neighborhoods are designed based on a single robot and multi robots, and a problem-oriented dynamic neighborhood strategy list is given for the collaborative employment bees stage. Additionally, the neighborhoods based on the critical robot are constructed to enhance the local exploitation capability. A novel collaborative following bee strategy is also integrated to help individuals trapped in the local optimum move to more ideal areas in the scout bee stage. Finally, comprehensive design experiments are carried out to determine optimal parameter configurations and demonstrate that CDABC has the better performance than the state-of-the-art algorithms.

1. Introduction

In Agriculture 4.0, the agriculture has entered an era of intelligence, unmanned [1,2] and smart farming [3]. As the hallmark of Agriculture 4.0, smart farming has revolutionized traditional farming methods by introducing intelligent agricultural robots. These robots are particularly instrumental in critical tasks such as harvesting and weeding, significantly enhancing efficiency and precision in agricultural practices [4]. However, the complexity of efficiently allocating and scheduling these robots remains a significant challenge [5,6]. Complexity challenges, as well as operational costs and efficiencies consideration motivate us to study the multiple agriculture robots task allocation and scheduling problem. The aim is to provide a solution that not only improves task allocation but also addresses the challenges of cost-effectiveness and operational efficiency. In smart farms, a fleet of n homogeneous agricultural robots are initially stationed at the depot, waiting instructions from the control system to perform allocated tasks. Each robot departs from the depot and returns to the depot after completing its allocated tasks [7]. The integration of these robots into agricultural practices has markedly improved productivity and saved labor [8]. In order to improve efficiency, it is necessary to explore efficient methods for high-quality task allocation and scheduling that take into account specific problem characteristics like robot location and task duration.

The multiple agricultural robots task allocation and scheduling (MRTAS) problem is a complex NP-hard problem [9]. It is often impractical to solve complex NP-hard problems in finite time using exact methods. Instead, heuristics and metaheuristics are increasingly recognized as effective methods for identifying optimal or near-optimal solutions in a reasonable time [9,10]. For solving this problem, we focus on the discrete artificial bee colony (DABC) algorithm, notable for its balanced global and local exploration capabilities and few control parameters [11]. It has been proved to be competitive with other population-based algorithms [11] and has been widely applied to scheduling problems, including vehicle scheduling [12,13] and shop scheduling [14,15] problems. The DABC is composed of employed bee, onlooker bee and scout bee stages. In each stage, appropriate strategies can be embedded according to the optimization objective and characteristics of the problem to improve the search ability. However, the application of DABC in agriculture weeding robot task allocation and scheduling is very limited. Considering the superior performance of DABC, we propose a collaborative discrete artificial bee colony (CDABC) for solving the multi-weeding-robots task allocation and scheduling

* Corresponding author.
E-mail address: 15628837665@163.com (Z. Miao).

https://doi.org/10.1016/j.knosys.2024.111474
Received 28 October 2022; Received in revised form 31 January 2024; Accepted 2 February 2024
Available online 5 February 2024
0950-7051/© 2024 Published by Elsevier B.V.

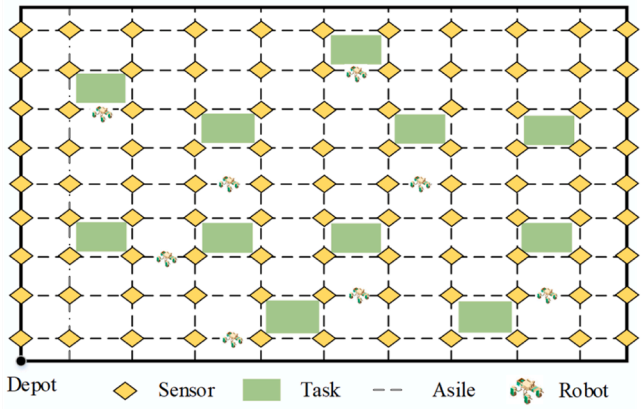


Fig. 1. The layout diagram of smart farm.

problem in a smart farm. The main contributions of this paper can be summarized as follows.

(1) Two construction heuristics based on the problem knowledge are proposed to provide high quality initial solutions for CDABC. A collaborative following bee strategy is developed to guide the bee to escape the local optimum, by preserving valuable information such as ever-improved individuals during the search process.
(2) A dynamic strategy list is introduced into the employed bee stage to help employed bees fully explore the solution space, and four solution neighborhoods are proposed in the onlooker bee stage based on the critical robot for further collaboratively local exploitation.
(3) Extensive experimental comparisons results demonstrate the effectiveness of the collaborative following bee strategy, and show the good performance of CDABC in solving the multi-weeding-robots task allocation and scheduling problem in a smart farm.

This paper is organized as follows. In Section 2, the related literature is reviewed. In Section 3, the formulation and description of problem is presented. Section 4 presents the proposed collaborative discrete artificial bee colony algorithm in detail. Section 5 gives the experimental design of parameter calibration and Section 6 provides a series of numerical experiments and comparisons. Section 7 concludes the paper and gives the outlines for the future research.

2. Related works

As there is limited existing research on the multi-weeding robots task allocation and scheduling problem, this section presents researches related to task allocation in analogous fields. It is worth noting that the majority of these research concentrates on allocation and does not encompass scheduling. Consequently, this section continues to use 'MRTA' from the original literature to denote the task allocation problem.

In 2004, Gerkey and Matari[16] proposed a classification method for the task allocation problem. From the perspective of robot capability, it can be divided into single-task robot and multi-task robot. From the perspective of task requirements, it can be divided into single-robot task and multi-robot task. From the perspective of real-time task allocation, it can be divided into instantaneous allocation system and time-extended allocation system. In 2005, the MRTA was listed as a special session of the international conference "Robots and Automation" and it has been gradually gaining attention from more scholars. Atay and Bayazit [17] introduced a mixed-integer linear programming model for allocating multiple heterogeneous robots to detect and control multiple regions of interest in an unknown environment. Gerkey and Mataric [18] proposed an auction-based task allocation system called MURDOCH, which relies on a resource-centric communication model. Jones et al. [19] improved the market-based task allocation method by incorporating regression-based learning to reduce overall penalties. Luo et al. [20] designed the highest-budget heuristic (HBH) for online MRTA and designed an online algorithm based on iterated greedy auction heuristic. However, these approaches use the central control communication unit and require more resources for cooperation, and the scalability is only guaranteed in small and medium-sized problems [21]. In contrast, in the nature-inspired optimization-based methods, the individuals are distributed, and there is no central control and global data in the system, which makes the system more robust. The optimization-based methods have shown some good characteristics, such as the ability of self-organization in unknown environment and the adaptive behavior produced by simple interaction between individuals. Consequently, researchers have increasingly focused their attention on these optimization-based methods [21]. The simulated annealing algorithm (SA) was combined with other heuristics to allocate a set of tasks to multiple processors in the computer system [22]. Bi et al. [23] designed an improved SA algorithm for solving the problem of charging multiple sensors. Jones et al. [24] adopted genetic algorithm to find solutions for time-expanded MRTA in disaster scenarios. Zhang et al. [25] presented an adaptive task allocation method for fully distributed mobile robots with identical functions in dynamic environments and built a task allocation hierarch, which the higher levels used a self-reinforcement learning model, and the lower levels used the ant colony algorithm (ACO) to organize task allocation. Zitouni and Maamri [26] solved the MRTA by a two-stage approach. In the first stage, they employed the firefly algorithm (FA) for global allocation, while in the second stage, they used the set theory properties for local allocation. A similar two-stage approach (FA-QABC) was also presented in [27], where the FA was used for global allocation, and the quantum artificial bee colony (QABC) algorithm was used for local allocation. Experimental results demonstrated that FA-QABC had advantages in allocation time in comparison with the approach proposed in [26]. Choudhury et al. [10] designed a stochastic conflict-based allocation (SCoBA) method for the MRTA under time constraints. SCoBA utilized dynamic programming to select the strategy for each agent to reduce the number of unsuccessful tasks. Ding et al. [28] designed an adaptive decay factor for the ACO algorithm to avoid deadlock. Necula et al. [29] adopted clustering methods to group and optimized solutions based on the ant colony system algorithm to minimize the maximum path cost. Xue et al. [30] studied a large-scale many-objective MRTA and proposed a hybrid many-objective competitive swarm optimization (HMaCSO) algorithm that utilized an adaptive meshing strategy (AGDS) and a mating pool selection method. Zhou et al. [5] studied task allocation, scheduling and planning in the design of the spot-welding production line, which is involved with multi-station and multi-robot. Several other studies employed optimization-based algorithms to address task allocation problems such as the reference [31–33]. In addition, Badreldin et al. [34] conducted a comprehensive comparative study to compare the optimization-based and market-based methods for solving the MRTA in several test scenarios. The results showed that the superiority of the optimization-based method in terms of optimizing configuration and computing time. Recently, Dai et al. [35] studied the multi-objective picking-robot task allocation and scheduling problem for agricultural picking scenarios and proposed a multi-objective discrete artificial bee colony algorithm (MDABC). Kang et al. [36] proposed a teaching-learning-based optimization algorithm (TLBO) to solve the problem of task allocation and scheduling for multiple weeding-robots considering pesticide residuals.

The literature review above indicates that while task allocation under various constraints has garnered significant attention, there remains a limited number of studies addressing both allocation and scheduling processes, particularly in the context of smart farms. Advanced heuristics and optimization methods were desired for a large

Table 1
The weeding time and coordinate of tasks.

	$Depot/J_0$	J_1	J_2	J_3	J_4	J_5
coordinate	(0,0)	(8,12)	(1,14)	(5,2)	(11,7)	(12,9)
weeding time	–	42	37	65	14	83

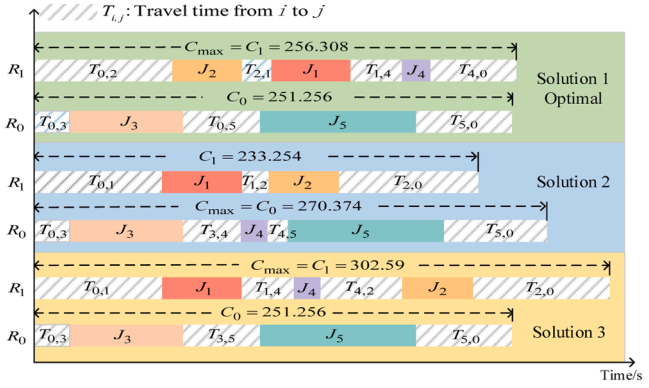


Fig. 2. A Gantt chart for the example instance.

number of robots in practical engineering applications [37]. Considering the superiority of the optimization-based methods to solve MRTA, this paper proposes a collaborative discrete artificial bee colony (CDABC) algorithm for solving the multiple weeding robots task allocation and scheduling problem.

3. Problem description and formulation

3.1. Problem description

In a typical smart farm located in Shouguang City, a comprehensive network of sensors, including GPS, temperature, humidity sensors, and smart cameras, is regularly arranged on the farm (as illustrated in Fig. 1). Each sensor unit in this network plays a crucial role in monitoring environmental conditions and crop growth. According to the statistics, there are nearly 1500 species of weeds in China, which has become a key factor affecting crop harvest and quality [11]. Given the significant challenge of weed control, farmers are increasingly eager to embrace robotic technology for effective weed management [38]. The sensors collect data on the growth of vegetables, as well as the distribution of pests and weeds. The control system allocates robots to perform weeding tasks based on the weed information collected by the sensors. The allocated robots depart from the depot and arrive at the task through the aisles. To ensure the efficient completion of work, it is necessary to optimize both the "task-to-robot" allocation and the "robot-to-task" scheduling. The process of task allocation and scheduling significantly affect the overall efficacy of the smart farm.

The importance of reasonable task allocation and task scheduling in smart farms can be illustrated by taking 5 weeding tasks and 2 robots (n=2, m=5) as an example. The weeding time and location of tasks are shown in Table 1, and the Gantt chart of the complete allocation and scheduling solution is shown in Fig. 2, where the Solution 1 is optimal. From Fig. 2, it is clear that allocating tasks to multiple robots using random allocation or geographic proximity (e.g., Fig. 2 (Solution 2 and Solution 3)) may produce solutions of uneven quality. In contrast, an allocation solution that considers task location and working time (problem characteristics) can greatly improve the solution quality (Fig. 2 (Solution 1)). Furthermore, by comparing the R_1 of Solution 1 and Solution 3, it becomes apparent that even if a good allocation is obtained during the task allocation stage, an unreasonable task scheduling could lead to a low-quality solution. Compared with Solution 2 and Solution 3, the maximum completion time of Solution 1 saves approximately 18.04 % and 5.5 %, and the load is more balanced among the robots. Therefore, a reasonable task allocation and scheduling can significantly improve production efficiency. With the goal of minimizing the maximum completion time, this paper studies the multi weeding-robots task allocation and scheduling problem in a smart farm and proposes an efficient task allocation method based on DABC algorithm.

3.2. Problem formulation

In this study, we address the problem of task allocation and scheduling in a smart farm, where a limited number of weeding robots (n) responsible for weeding tasks (m) in multiple areas. Assuming the number of robots is much smaller than tasks, i.e. $n \ll m$. Allocating one task to more than one robot would lead to infeasible allocations. The objective is to find a set of optimal task allocation and task sequences for each robot to optimize maximum completion time. In addition, without loss of generality, several assumptions are made as follows:

(a) All robots run normally, without downtime or malfunction.
(b) All robots are homogeneous.
(c) All robots start from the depot and return to the depot after completing the task.
(d) The robot moves at a constant speed with a known speed.

Notation:

R	$R = \{R_1, R_2, \ldots R_n\}$; The set of robots, which contains n robots.
J	$J = \{J_0, J_1, J_2, \ldots J_m\}$; The set of depot and task nodes, including m places waiting to be weeded. J_0 is the depot.
v	Robot speed.
k	Robot index.
i,j,l	Task index.
x_i, y_i	Coordinate values of depot and task point in set J.
t_i	Weeding time required for task point i, and $t_0 = 0$.
$D_{i,j}$	The distance from task J_i to task J_j.
$\Pi = \{\pi_1, \pi_2, \ldots \pi_n\}$	A complete solution.
$\pi_k = \{J_1, J_2, \ldots J_{u_k}\}$	The sequence of tasks performed by robot R_k,u_k is the total number of tasks performed by robot R_k.
C_k	The time required for robot R_k to return to the depot after performing all its assigned tasks.
C_{max}	Completion time of the last task.

Decision variables:

Y_{ik}	If the robot R_k performs task J_i, then $Y_{ik} = 1$, else $Y_{ik} = 0$.
X_{ijk}	If the robot R_k travels from task J_i to task J_j, then $X_{ijk} = 1$, else $X_{ijk} = 0$.
S_{ijk}	If robot R_k travels from task J_i to task J_j, its value is the number of tasks that robot R_k has performed before task J_i, otherwise it is 0.

Objective and constrains:

$$Min\ (max(C_k)) \tag{1}$$

$$\sum_{k=1}^{n} Y_{ik} = 1, \forall i \backslash \{0\} \tag{2}$$

$$Y_{0k} = 1,\ k = 1, 2, \ldots n \tag{3}$$

$$\sum_{j=0, j\neq i}^{m} X_{i,j,k} = Y_{ik},\ i = 0, 1, \ldots m, \forall k \tag{4}$$

$$\sum_{i=0, i\neq j}^{m} X_{i,j,k} = Y_{jk},\ j = 0, 1, \ldots m, \forall k \tag{5}$$

$$S_{ijk} \le X_{ijk} \cdot \left(\sum_{l=0}^{m} Y_{lk} - 1 \right), \ \forall k, i \ne j \tag{6}$$

$$\sum_{j=0}^{m} S_{ijk} - \sum_{j=0}^{m} S_{jik} = 1, \forall i \backslash \{0\}, \forall k \tag{7}$$

$$C_k = \sum_{i=0}^{m} \sum_{j=0, j \ne i}^{m} \left(X_{i,j,k} \cdot \frac{fabs(x_j - x_i) + fabs(y_j - y_i)}{v} \right) + \sum_{i=0}^{m} (Y_{ik} \cdot t_i), \ \forall k \tag{8}$$

Eq. (1) is the objective function, which minimizes the completion time of the last robot to complete the task. Eq. (2) means that each task can only be completed by one robot. Eq. (3) ensures that each robot starts from the depot. Eqs. (4) and (5) are the associating constraints that ensures the balance of inflow and outflow. Eq. (4) guarantees that if robot R_k performs task J_i, then robot R_k performs task J_i followed by a unique successor task (the successor task cannot be J_i), while Eq. (5) assures that if robot R_k performs task J_j, then robot R_k has a unique precursor task (which cannot be itself) before it performs task J_j. Eqs. (6) and (7) ensure that no sub tour appears. Finally, the completion time of robot R_k is calculated by Eq. (8).

4. Proposed collaborative discrete artificial bee colony algorithm

The proposed CDABC framework introduces a novel collaborative following bee strategy, aimed at enhancing collaborative interaction among individuals. The following behavior is selectively performed during the employed bee stage and onlooker bee stage. To enhance the global search capability of bees, a problem-oriented dynamic strategy list is adopted in the employed bee stage. In the onlooker bee phase, we design several neighborhood operators centered on the critical robot to provide in-depth local exploitation. During the scout bee stage, the collaborative following bees guide individuals trapped in a local optimum towards a more desirable area. The pseudo code of the proposed CDABC is shown in Procedure 1.

Procedure 1: CDABC(Psize, r_{cal}, λ)

1. Initialization stage (Generate *Psize* individuals using the initialization method in Section 4.2)
2. **while** *the stopping criterion is not satisfied*
3. The collaborative employed bee stage() // Procedure 8 in Section 4.5
4. The collaborative onlooker bee stage()// Procedure 9 in Section 4.6
5. The collaborative scout bee stage()// Procedure 10 in Section 4.7
6. Find the best individual and update population
7. **endwhile**
8. **Output** π_{best}

4.1. Solution representation

In order to determine a feasible solution for the studied problem, it needs to allocate weeding tasks for each robot and arrange the execution orders of each robot (i.e. task scheduling). In CDABC, a solution is represented by a 2-dimensional list, where the first dimensional represents n robots and the second dimensional represents the tasks allocated to each robot. A solution representation for the example (Table 1) is shown as $\Pi = \{\pi_1, \pi_2\}$, $\pi_1 = \{J_3, J_5\}$, $\pi_2 = \{J_2, J_1, J_4\}$, where $u_1 = 2$, $u_2 = 3$.

4.2. Proposed heuristics and initialization

To get good initial solutions, this paper designs two heuristics based on problem characteristics. The first heuristic is based on the rule of dominant thought (called **Heu_Dominant**). For tasks, there are two basic attributes: weeding time ($attribute_{i,1}, i \in J$) and distance ($attribute_{i,2}, i \in J$). First, two attributes are normalized according to Eq. (9). Second, all tasks are divided into two categories. If the weeding time of the task dominates the distance (from the start point), the task is added to *Group* 1, that is, $D_{0j}^{norm} \ge t_j^{norm}$. Otherwise, the task is added to *Group* 2. For the tasks in *Group* 1, the robot spends more time on the road than the weeding time. The tasks of *Group* 1 are allocated to the best position one by one in the non-decreasing order of $D_{0,j}$, $j \in$ *Group* 1. For the tasks in *Group* 2, the robot spend more time in weeding operation, so the tasks with larger weeding time are allocated to the best position in priority. The pseudo code is shown in Procedure 2.

$$\frac{\exp(attribute_{i,t})}{\exp(attribute_{i,t})}, \ i \in J, \ attribute_{i,t} \in \{attribute_{i,1}, attribute_{i,2}\} \tag{9}$$

Procedure 2: Heu_Dominant()

1. **Begin**
2. **for** $j = 1$ to m
3. calculate the normalized values for the two attributes according to Eq. (9)
4. **if** $D_{0j}^{norm} \ge t_j^{norm}$
5. add the task j to *Group* 1
6. **else**
7. add the task j to *Group* 2
8. **endif**
9. **endfor**
10. **for** j in *Group* 1
11. sort the task in *Group* 1 in the non-decreasing order of $D_{0,j}$
12. insert the task in *Group* 1 to the best position in turn
13. **endfor**
14. **for** j in *Group* 2
15. sort the task in *Group* 2 in the non-increasing order of t_j
16. insert the task in *Group* 2 to the best position in turn
17. **endfor**
18. **end**

The second heuristic only considers the weeding time (called **Heu_Time**). In the Heu_Time, we give priority to tasks with greater weeding time, that is, tasks are allocated the best position in the non-increasing order of $\sum_{i=1}^{m} attribute_{i,1}$.

As a population-based metaheuristic, it is very important for CDABC to evolve from a population with high level of quality and diversity. Therefore, we use the proposed heuristics (Heu_Dominant and Heu_Time) to generate 2 individuals to ensure the quality of the population. At the same time, the rest *PSize*-2 individuals are generated randomly to ensure the diversity of the population.

4.3. Solution neighborhoods

According to existing literature, insertion and exchange are two effective and commonly used neighborhood operators in scheduling problems [38]. Based on this, we adapt these operations to specifically address the weeding robot task allocation and scheduling problem, enabling simultaneous consideration of robot allocation and task scheduling. The insert operator extracts a task from the original position and reinserts it into another position, while the exchange operator needs to select two tasks to exchange. Considering the multi-robot characteristics of the studied problem, we redesign the insert operator and the exchange operator for single robot and multi robots.

The first neighborhood operator is single robot insertion, defined as the SR-Insert. It performs n rounds until there is no improvement, hoping that each robot can be explored. In each round, we randomly select a robot, then randomly select a task from the selected robot and insert it into the best position. The pseudo code is shown in Procedure 3.

The second neighborhood operator is single robot exchange, defined as the SR-Exchange. We randomly select two tasks from a robot and exchange them. If the exchange results in improvement, the neighborhood solution is accepted. This process is repeated n times until no improvement occurs. The pseudo code is shown in procedure 4.

Next, we consider the neighborhood actions between multiple robots, which are multi-robot insertion and multi-robot exchange, defined

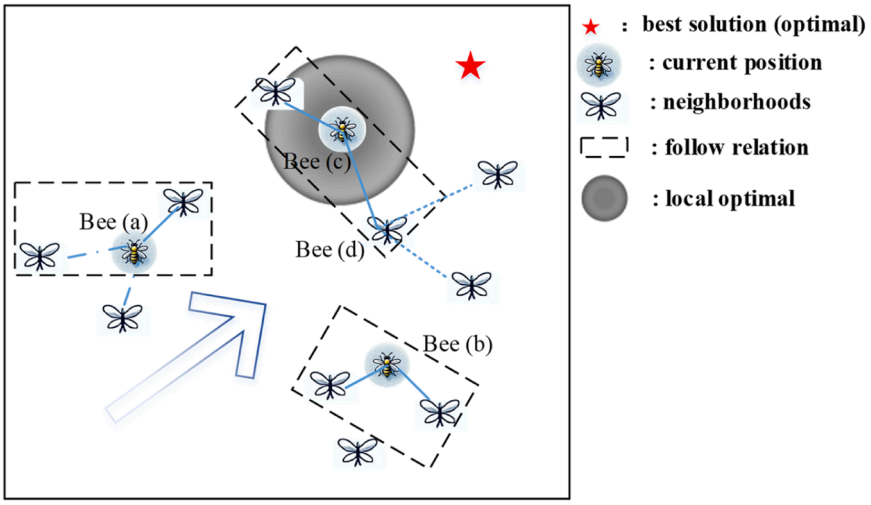


Fig. 3. Example of the collaborative following bee strategy.

as the MR-Insert and the MR-Exchange. In MR-Insert, two robots, R_p and R_q, are randomly selected, where $q \neq p$. Assuming robot R_p has a larger completion time. Randomly select a task from R_p and insert it into the best position of R_q. If the new completion time of R_q is less than the original completion time of R_q, the neighborhood solution is accepted. MR-Insert executes n rounds until there is no improvement. Once the quality of solution is improved, it executes new n round extraction and insertion, which gives each robot at least one opportunity to be explored. The pseudo code is shown in Procedure 5. For MR-Exchange, two robots, R_p and R_q, are randomly selected, where the relatively larger completion time is denoted as C_{larger}. Then two tasks from R_p and R_q are randomly selected and exchanged. If the new relatively larger completion time is better than C_{larger}, the new neighborhood solution is accepted. Similarly, we repeat the above process n times to obtain a better neighborhood. The pseudo code is shown in Procedure 6.

Procedure 3: SR-Insert()

1. $Flag$ = true;
2. **while** $Flag$
3. $Flag$ = false;
4. **for** $i = 1$ to n
5. $R_{sel} \leftarrow$ select a robot randomly
6. $J_{sel} \leftarrow$ select a task from selected robot randomly
7. $\pi'_{sel} \leftarrow$ reinsert J_{sel} to the best position
8. **if** $C'_{sel} < C_{sel}$
9. $Flag$ = true; $\pi_{sel} = \pi'_{sel}$;break;
10. **endif**
11. **endfor**
12. **endwhile**

Procedure 4: SR-Exchange()

1.$Flag$ = true;
2.**while** $Flag$
3. $Flag$ = false;
4. **for** $i = 1$ to n
5. $R_{sel} \leftarrow$ select a robot randomly
6. $J_{selx}, J_{sely} \leftarrow$ select two different tasks from selected robot randomly
7. $\pi'_{sel} \leftarrow$ exchange J_{selx} and J_{sely}
8. **if** $C'_{sel} < C_{sel}$
9. $Flag$ = true; $\pi_{sel} = \pi'_{sel}$;break;
10. **endif**
11. **endfor**
12. **endwhile**

Procedure 5: MR-Insert()

(continued on next column)

(*continued*)

Procedure 5: MR-Insert()

1.$Flag$ = true;
2.**while** $Flag$
3. Flag = false;
4. **for** $i = 1$ to n
5. $R_p, R_q \leftarrow$ select two different robots randomly
6. **if** $C_q < C_p$
7. $C_{larger} = C_p$, $C_{smaller} = C_q$, larger = p, smaller = q
8. **else**
9. $C_{larger} = C_q$, $C_{smaller} = C_p$, larger = q, smaller = p
10. **endif**
11. $J_{largersel} \leftarrow$ select a task from selected robot randomly
12. $\pi'_{smaller} \leftarrow$ reinsert $J_{largersel}$ to the best position of $R_{smaller}$
13. $\pi'_{larger} \leftarrow$ remove $J_{largersel}$ from R_{larger}
14. **if** $C'_{smaller} < C_{larger}$
15. Flag = true; $\pi_{smaller} = \pi'_{smaller}$; $\pi_{larger} = \pi'_{larger}$; break;
16. **endif**
17. **endfor**
18. **endwhile**

Procedure 6: MR-Exchange()

1: $Flag$ = true;
2: **while** $Flag$
3: Flag = false;
4: **for** $i = 1$ to n
5: $R_p, R_q \leftarrow$ select two different robots randomly
6: **if** $C_q < C_p$
7: $C_{larger} = C_p$, $C_{smaller} = C_q$, larger = p, smaller = q
8: **else**
9: $C_{larger} = C_q$, $C_{smaller} = C_p$, larger = q, smaller = p
10: **endif**
11: $J_{selx} \leftarrow$ select a task from R_p randomly and remove it
12: $J_{sely} \leftarrow$ select a task from R_q randomly and remove it
13: $\pi'_p, \pi'_q \leftarrow$ exchange J_{selx} and J_{sely}
14: **if** $C'_p < C'_q$
15: $C'_{larger} = C'_q$
16: **else**
17: $C'_{larger} = C'_p$
18: **endif**
19: **if** $C'_{larger} < C_{larger}$
20: Flag = true; $\pi_p = \pi'_p$; $\pi_q = \pi'_q$; break;
21: **endif**
22: **endfor**
23: **endwhile**

4.4. Collaborative following bee strategy

It is a common problem of metaheuristic that individuals are trapped in local optimum or meet premature convergence, and DABC is not exception [39]. To address this problem, we introduce the collaborative following bee strategy (CFS) in CDABC, aimed at selectively retaining potential solutions and mitigate the tendency of bees to ignore potential areas due to their greedy characteristics. Considering the time and space costs, each bee is limited to have a maximum of two collaborative following bees, denoted as *Co-followSize*=2. Specifically, collaborative following bees are generated in the collaborative employed bee stage and collaborative onlooker bee stage. In the scout bee stage, it assists the bee jump out of the local optimum. In the employed bee phase, bees first explore and evaluate the neighborhood of their current positions. Upon discovering a better solution, the bee greedily flies to this neighboring solution. At the same time, the current bee is marked as the collaborative following bee (Fig. 3, Bee (a)), and establishes a following relationship with the new solution. Solutions that yield superior neighborhoods possess excellent genes, thereby guiding the bees toward an optimal region. If the generated neighborhoods are worse than the current one, the best two are selected to update collaborative following bees (Fig. 3, Bee (b)). The collaborative following bee strategy is visually shown in Fig. 3, where the red pentagram is the target (optimal solution). The current solution and its neighborhoods are connected by blue line segments (including dashed and solid lines), indicating their following relationship. However, the actual following relationship is also determined by the restriction of *Co-followSize*. In addition, if the bee falls into a local optimum during the search (Fig. 3, Bee (c)), its collaborative following bees are likely to help it jump out of the local optimum and explore more promising regions (Fig. 3, Bee (d)).

Procedure 7: StrategyList()

```
1:  r_max = r_0 + r_cal, the length of Strategy_List = Psize*(1 + r_max), l = 0
2:  while l < the length of Strategy_List
3:    Neighborhood = rand() % 4
4:    switch (Neighborhood)
5:      case 0: Strategy_List[l] = SR-Insert();
6:      case 1: Strategy_List[l] = SR-Exchange();
7:      case 2: Strategy_List[l] = MR-Insert();
8:      case 3: Strategy_List[l] = MR- Exchange ();
9:    endswitch
10:   l++
11: endwhile
```

4.5. Collaborative employed bee stage

In this stage, the employed bees mainly perform global search and actively search new food sources [33,39]. Selecting appropriate neighborhood operators during this phase is vital for the efficacy of the algorithm. In traditional DABC, the number of space explorations (r) remains constant for each employed bee throughout the iteration. This fixed value of r inevitably results in a limited search space. However, as shown in previous studies [40], a dynamic variation of r value within a given range can yield more extensive and diverse search areas compared to the fixed r value, which can have a positive effect on CDABC. Therefore, in CDABC, we randomly generate r value in the range [1, r_{max}], where $r_{max} = r_0 + r_{cal}$. Here, r_{cal} is an experimentally calibrated parameter, and r_0 is a positive number calculated by the average robot load (Eq. (10)). In addition, we introduce a neighborhood strategy list for each employed bee to further ensure the diversity. This strategy list is composed of the four neighborhood operators proposed in Section 4.3 and has a length of *Psize**(1+ r_{max}). Each element of the strategy list is a random permutation of the four neighborhood operators. The pseudo code of strategy list is shown in Procedure 7. The bees explore the neighborhoods based on the strategy list, and then update the collaborative following bees. The framework for this stage is given in Procedure 8.

$$r_0 = \begin{cases} 1, & m/n < 15 \\ 2, & 15 \le m/n < 25 \\ 3, & m/n \ge 25 \end{cases} \tag{10}$$

Procedure 8: The collaborative employed bee stage()

```
1.Begin
2.  r_max = r_0 + r_cal, ncount_List = 0
3.  for ind = 1 to Psize
4.    r = 1 + rand % r_max
5.    for rt = 0 to r
6.      π' ← Strategy_List[ncount_List]
7.    endfor
8.    ncount_List++
9.    if C_max(π') < C_max(π) // the collaborative following bee strategy
10.     Co-following-bee ←π, π = π'
11.   else
12.     if the size of Co-following-bee < Co-followSize
13.       add π' to Co-following-bee without repetition
14.     else
15.       discard the worst bee in Co-following-bee and add π' to Co-following-
    bee
16.     endif
17.   endif
18.   Update the best solution
19. endfor
20.end
```

4.6. Collaborative onlooker bee stage

In the proposed CDABC, the tournament algorithm is used to select the individual for the next operation. For collaborative onlooker bees, focusing on the robot with the maximum completion time (defined as the critical robot) is the key to optimizing the system efficiency. Based on the critical robot idea, we introduce three neighboring operators: Criticalrobot-MR-Insert, Criticalrobot-MR-Exchange, and Criticalrobot-SR-Exchange. They are similar to the MR-Insert, MR-Swap, SR-Swap proposed in Section 4.3. The difference is that the robot is no longer randomly selected, but at least one robot is the critical robot. In this way, individuals can be improved in a more targeted manner and the neighborhoods of the current solution can be fully exploited.

In the traditional DABC framework, the current solution is updated only when a neighborhood with a superior fitness value is found. If no improvement is found, the new neighborhood is directly discarded [38]. In CDABC, we consider the update of collaborative following bees and the improvement of the entire population quality. If the quality of the neighborhood is better than the current solution, the current one is updated, and the CFS is triggered. Otherwise, if this neighborhood solution is better than the worst individual in the population, the worst individual is replaced because a better solution must carry more useful information. Only when none of the above conditions is satisfied, the neighborhood solution is abandoned. This approach serves a dual purpose: it not only improves the overall quality of the population, but also accelerates the evolution process by avoiding ineffective searches (Because it's a waste to expend effort to generate a new neighborhood solution and just give up on it without any operation). The pseudo code of collaborative onlooker bee stage is shown in Procedure 9.

Procedure 9: The collaborative onlooker bee stage()

```
1.Begin
2.    for ind = 1 to Psize
3.      worst_ind ← select the worst individual from the population
4.      Ind_Tournament ← select an individual randomly by the tournament
    algorithm
5.      perform the Criticalrobot-MR-Insert to Ind_Tournament
6.      perform the Criticalrobot-MR-Exchange to Ind_Tournament
7.      π' ← perform the Criticalrobot-SR-Exchange to Ind_Tournament
8.      if C_max(π') < C_max(π) // the collaborative following bee strategy
9.        clear the Co-following-bee
```

(continued on next page)

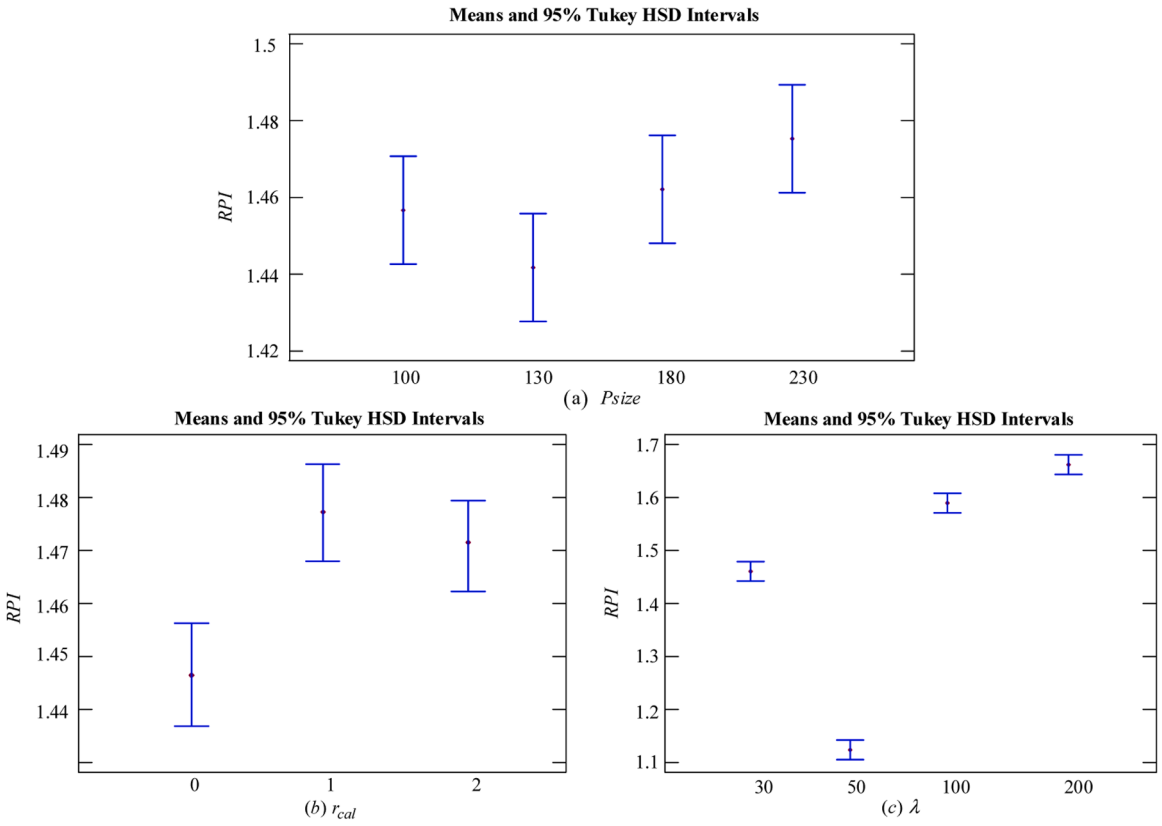


Fig. 4. Means plots and 95 % Tukey HSD interval for the parameters of the CDABC.

(*continued*)

Procedure 9: The collaborative onlooker bee stage()

```
10.        Co-following-bee ←π, π = π'
11.      endif
12.      if C_max(π') < C_max(worst_ind)
13.        worst_ind = π'
14.      endif
15.    endfor
16.end
```

4.7. Collaborative scout bee stage

In the scout bee stage, when an individual is not updated after a specified number of iterations, it should be replaced by a more promising individual [41–43], rather than a randomly generated individual as in traditional DABC. In the proposed CDABC, we utilize the collaborative following bees to provide the possibility for the unimproved individual to jump out of the local optimum, because these bees retain some information from the unimproved individual but maintain a certain distance from the original unimproved individual. If an individual fail to improve for more than λ consecutive iterations, we apply the SR-Insert, MR-Insert, SR-Exchange and MR-Exchange to perturb the collaborative following bee and update the unimproved individual. Here, the parameter λ will be calibrated in Section 5.2. The pseudo code of collaborative scout bee stage is presented in Procedure 10.

Procedure 10: The collaborative scout bee stage()

```
1.Begin
2.      Flag = false
3.      for ind = 1 to Psize
4.        Flag ← whether individual ind has not been updated for more than λ times
5.        if Flag // the collaborative following bee strategy is triggered
6.          co_ind ← select an individual randomly from the collaborative following
  bees
```

(*continued on next column*)

(*continued*)

Procedure 10: The collaborative scout bee stage()

```
7.          perform the SR-Exchange to the co_ind
8.          perform the MR-Exchange to the co_ind
9.          perform the SR-Insert to the co_ind
10.         π_ind ← perform the MR-Insert to the co_ind
11.       endif
12.     endfor
13.end
```

5. Experimental calibration

In this section, we first generate the test instances set based on the actual scenario of smart vegetable greenhouses in Shouguang. Then, the proposed CDABC is calibrated by Design of Experiments (DOE) and analysis of variance (ANOVA), which are common experimental analysis methods for combinatorial optimization problems [42,43].

5.1. Test instances

Currently, the Shouguang greenhouse industry has widely adopted sixth and seventh-generation greenhouses, and integrated agricultural robots for tasks such as inspection, weeding, and harvesting. These technological integrations have significantly enhanced operational efficiency, saved labor, and reduced costs. The sixth-generation greenhouse is approximately 100 m in length and 13 m in width, while the seventh-generation greenhouse is approximately 210 m in length and 21 m in width. Therefore, we generate both small-scale and large-scale instances based on the two sizes of greenhouse. The number of tasks (m) to be weeded in the small-scale instances is set to $m = \{45, 55, 65, 75\}$ and $m = \{80, 90, 100, 110\}$in the large-scale instances. The number of agricultural robot $n = \{2, 3, 4, 5, 6, 7\}$ and the weeding time of each task is randomly distributed at [10,100].

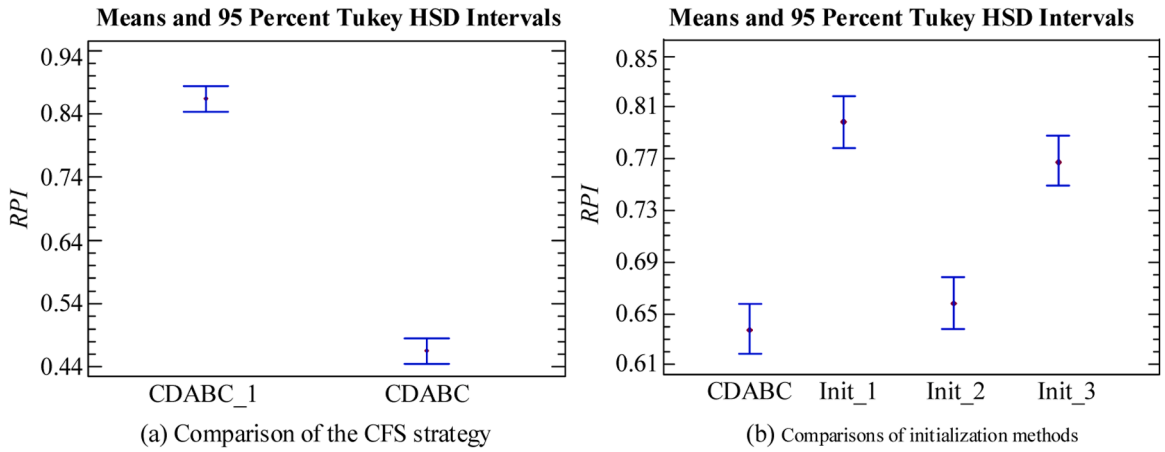


Fig. 5. Means plot and 95 % Tukey HSD interval of efficiency analysis.

Table 2
Results of Wilcoxon paired signed test ($\alpha = 0.05$).

Comparisons	R^+	R^-	Bonding Value	*p-value*
CDABC vs. CDABC_1	1824	571	5	0.00
CDABC vs. Init_1	1693	699	8	0.00
CDABC vs. Init_2	1370	1016	14	0.39
CDABC vs. Init_3	1599	792	9	0.00

Table 3
The parameters of all comparison algorithms.

Algorithms	Parameters
SA	$T = 1.0$, $\alpha 1 = 0.9$
QABC	$Psize = 100$, $gen = 40$
ACS	$Psize = 10$, $\alpha 2 = 0.6$, $\beta = 0.8$, $\rho = 0.1$, $q = 0.7$
IABC	$Psize = 30$, $FS = 1$
DABC	$Psize = 50$, $l = 600$, $r = 80$, $\tau = 25$
MDABC	$Psize = 30$, $TS = 5$, $g = 1$
TLBO	$Psize = 180$, $\varphi = 12$, $Pc = 0.9$, $Pm = 0.2$

T: initial temperature; $\alpha 1$: cooling coefficient; gen: quantum genetic algorithm execution times; $\alpha 2$: pheromone importance factor; β: heuristic function importance factor; ρ: pheromone volatilization factor; q: state transition probability; *FS*: the number of food source; *l*: consecutive iterations; *r*: replications in onlooker bee phase; τ: the performed times of insertion and swap operators; *TS*: tournament selection size; *g*: interval generation of local search; φ: local search operator execution times; *Pc*: crossover probability; *Pm*: mutation probability.

Table 4
Results of ANOVA for algorithms.

Source	Sum of Squares	Df	Mean Square	F-Ratio	P-Value
MAIN EFFECTS					
A:Algorithm	3.3669E6	7	480,986.	4298.19	0.0000
RESIDUAL	1.07339E6	9592	111.904		
TOTAL (CORRECTED)	4.44029E6	9599			

5.2. Parameters calibration

The proposed CDABC contains three parameters, *Psize*, r_{cal} and λ. Based on the existing literature and our past experiments, we carefully determine the ranges of the parameters: *Psize* at four levels 100, 130, 180, 230; r_{cal} at three levels: 0, 1, 2 and λ at four levels: 30, 50, 100, 200. Thus, we obtain a total of 64 parameter combinations ($4 \times 4 \times 4 = 64$). To calibrate the proposed CDABC, we generate 48 calibration

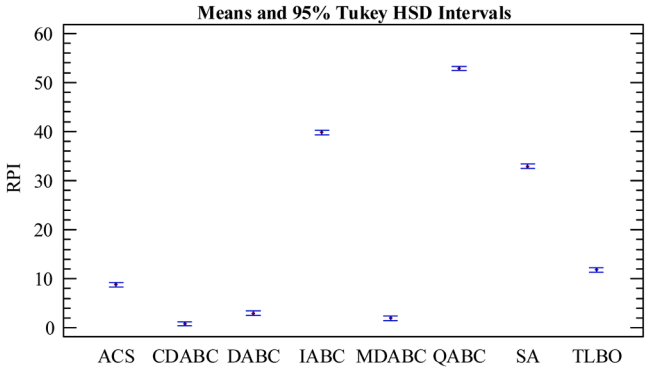


Fig. 6. The means plot of the compared algorithms.

instances with $n = \{2, 3, 4, 5, 6, 7\}$, $m = \{45, 55, 65, 75, 80, 90, 100, 110\}$. For each calibration instance, ten times independently replicates are performed. All algorithms are coded through C++ and performed in the same Intel (R) Core (TM) i7–9700 CPU @ 3.60 GHz, 4.00 GB RAM, 64-bit Windows 7 professional operating system. The stopping criterion of the algorithms is set to a predefined CPU time

$20 \times n \times m$ milliseconds. For CDABC, a total of 640 results ($64 \times 10 = 640$) are obtained for each instance. To better analyze these results, we use the relative percentage increase (*RPI*) as the response variable, which is calculated using Eq. (11).

$$RPI = \frac{C_{max}^{alg} - C_{max}^{*}}{C_{max}^{*}} * 100 \tag{11}$$

where C_{max}^{alg} is a solution and C_{max}^{*} is the best solution among the 640 results. We generate a total of 30,720 *RPI* values ($640 \times 48 = 30720$) for CDABC.

All results are analyzed by ANOVA, where the number of robot (n) and the number of task (m) are considered as non-controllable factors. Due to limited space, the results of the ANOVA analysis are briefly commented. In order to determine the best parameter level, we focus on the means plot with 95 % Tukey HSD interval obtained by ANOVA technology. It can be seen from Fig. 4(a) that the difference between different *Psize* is not significant, but $Psize = 130$ gives the best result. The other two parameters have significant differences, and the algorithm reaches its best when $r_{cal} = 0$ and $\lambda = 50$, as shown in Fig. 4(b) and(c). Therefore, the parameters of CDABC are set as follows: $Psize = 130$, $r_{cal} = 0$ and $\lambda = 50$.

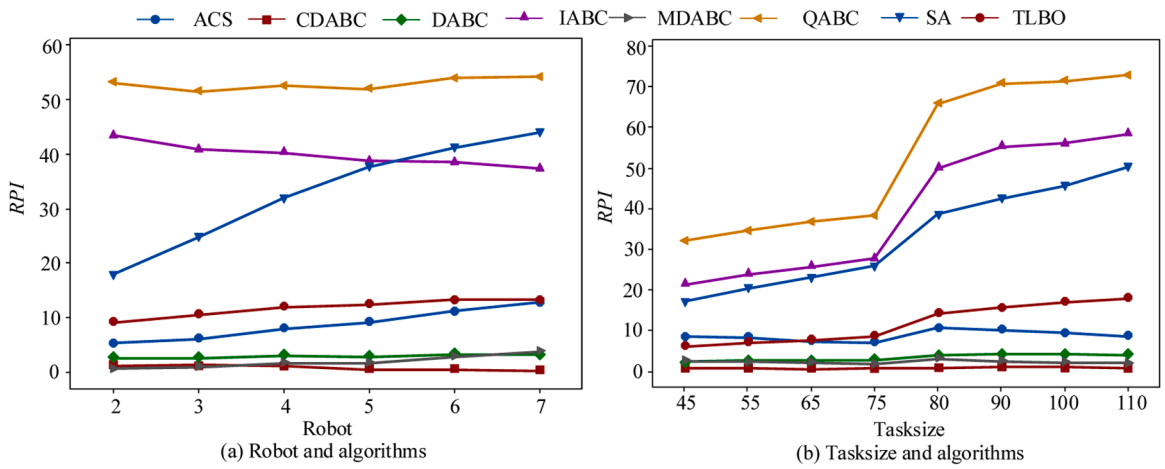


Fig. 7. The means plots of interaction with 95 % Tukey HSD intervals.

Table 5
Friedman-test results of eight algorithms.

Algorithms	Mean Rank (Small-Scale)			
	m = 45	m = 55	m = 65	m = 75
ACS	4.82	4.69	4.46	4.21
CDABC	**1.37**	**1.35**	**1.31**	**1.43**
DABC	2.42	2.47	2.6	2.81
IABC	6.68	6.58	6.55	6.52
MDABC	2.21	2.63	2.58	1.76
QABC	7.98	7.97	7.85	7.93
SA	6.33	6.39	6.44	6.55
TLBO	4.19	4.31	4.51	4.79
CN	150	150	150	150
p-value	0.00	0.00	0.00	0.00
Crit. Diff $\alpha = 0.05$	0.76	0.76	0.76	0.76

Table 6
Friedman-test results of eight algorithms.

Algorithms	Mean Rank(Large-Scale)			
	m = 80	m = 90	m = 100	m = 110
ACS	4.13	4.05	4.01	4
CDABC	**1.28**	**1.38**	**1.4**	**1.43**
DABC	2.7	2.72	2.79	2.69
IABC	6.61	6.69	6.69	6.57
MDABC	2.61	2.67	2.59	2.62
QABC	7.99	8	7.89	7.88
SA	6.31	6.26	6.33	6.35
TLBO	4.72	4.8	4.82	4.85
CN	150	150	4.98	150
p-value	0.00	0.00	0.00	0.00
Crit. Diff $\alpha = 0.05$	0.76	0.76	0.76	0.76

6. Computational evaluation

In this section, the effectiveness of the proposed heuristics and the collaborative following bee strategy are analyzed. Then, we evaluate the proposed CDABC through extensive experimental comparisons. We regenerate test instances for each combination of $n = \{2, 3, 4, 5, 6, 7\}$, $m = \{45, 55, 65, 75, 80, 90, 100, 110\}$ and repeat each combination 5 times. Therefore, we generate a total of 270 test instances. As in Section 5.2, all experiments are performed in the same environment, and *RPI* is adopted as the performance measure. For each instance, we repeat 10 times independent to obtain statistical results. When the maximum elapsed CPU time of $20 \times n \times m$ milliseconds is met, the algorithm is stopped.

Table 7
Friedman-test results of eight algorithms.

Algorithms	Mean Rank					
	n = 2	n = 3	n = 4	n = 5	n = 6	n = 7
ACS	4.2	4.13	4.23	4.34	4.39	4.49
CDABC	1.75	1.58	**1.3**	**1.23**	**1.13**	**1**
DABC	2.83	2.8	2.79	2.82	2.61	2.41
IABC	7	6.99	6.87	6.51	6.34	6.17
MDABC	**1.43**	**1.56**	2.13	2.09	2.26	2.6
QABC	8	8	7.99	7.95	7.93	7.91
SA	5.99	6.02	6.14	6.54	6.74	6.93
TLBO	4.82	4.87	4.77	4.66	4.62	4.5
CN	200	200	200	200	200	200
p-value	0.00	0.00	0.00	0.00	0.00	0.00
Crit. Diff $\alpha = 0.05$	0.659	0.659	0.659	0.659	0.659	0.659

6.1. Efficiency analysis

In this section, we evaluate the effectiveness of the initialization method and the collaborative following bee strategy. The related parameters of two comparison experiments are set at the best level of Section 5.2, and the same stop criterion are adopted. For each set of experimental results, we analyze their differences by ANOVA technology and the Wilcoxon paired signed test. The Wilcoxon paired signed test, a non-parameter test method, is utilized to detect significant differences between CDABC and other algorithms [44].

The first set of comparison experiment (CDABC and DABC_1) is to test the collaborative following bee strategy, where DABC_1 does not adopt the CFS and the other settings are identical to CDABC. As shown in Fig. 5(a), there is a significant difference between CDABC and DABC_1. CDABC has a lower *RPI* value, which indicates CDABC with the CFS obtains better results. In addition, the Wilcoxon test results are presented in Table 2, where R^+ represents the positive ranks, and R^- represents the negative ranks. For each pair, the sum of R^+, $R^-$and bonding values is equal to the total case numbers. If *p-value* is less than $\alpha = 0.05$, it indicates that the pair algorithms have a significant difference. From Table 2, we can see that CDABC and DABC_1 are significant different due to the *p-value* is less than 0.05. The above analysis results demonstrate the effectiveness of the proposed CFS. This is because collaborative following bees not only retain some information about individuals, but also assist individuals to jump out of the local optimum.

The second set of comparison experiment is to test the initialization method, where Init_1 does not adopt the proposed heuristics, Init_2 just does not adopt the Heu_Dominant, and the Init_3 just does not adopt the Heu_Time. The other settings are identical to CDABC. As can be seen from Fig. 5(b), the initialization method proposed in this paper achieves

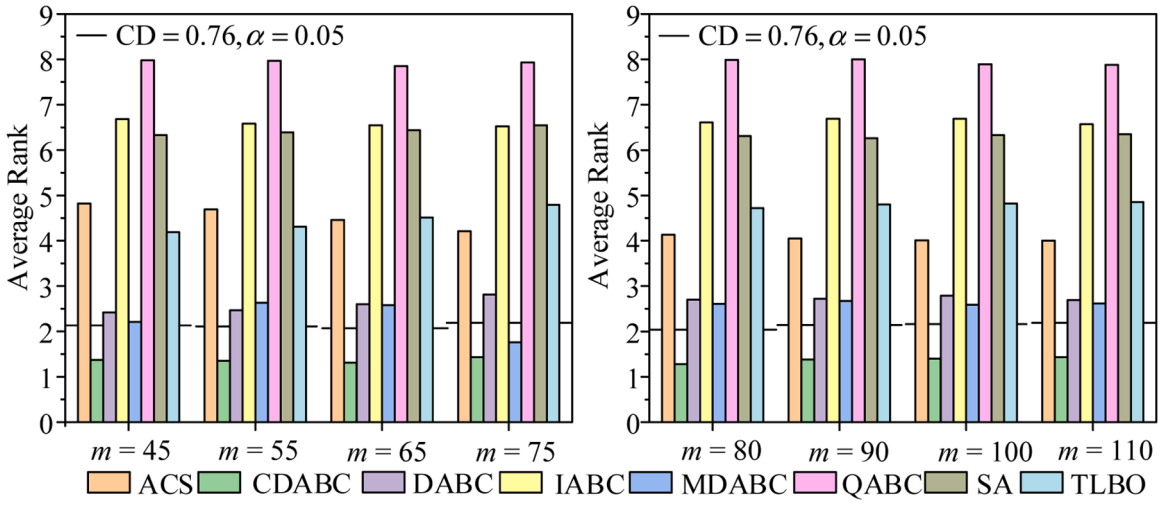


Fig. 8. Friedman-test results at different tasks.

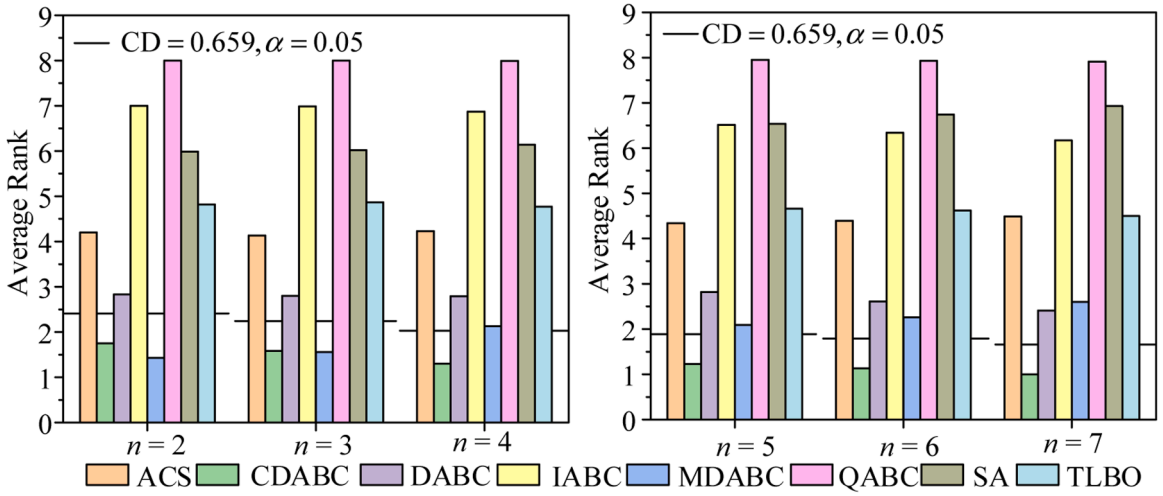


Fig. 9. Friedman-test results at different robots.

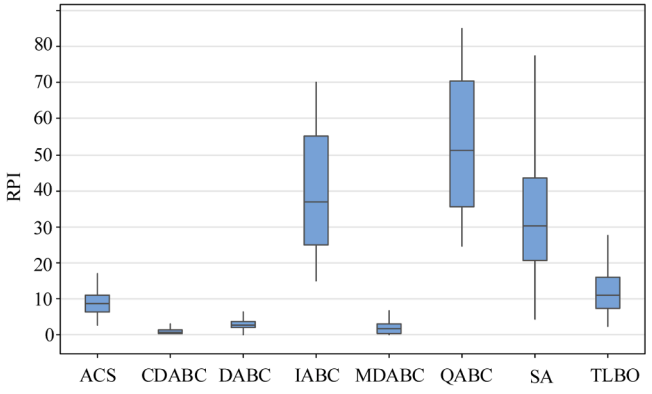


Fig. 10. The box plot for comparison algorithms.

the best results. CDABC is significantly better than Init_1 and Init_3 within the 95 % confidence interval. As can also be seen from Table 2, for each pair of comparisons, *p-value* is equal to zero, indicating a significant difference between CDABC and the other algorithms. Although the difference between CDABC and Init_2 is not significant due to *p-value* > 0.05, CDABC still outperforms Init_2. This is attributed to the use of multiple heuristics, which helps to maintain the diversity of individuals in the initial solution. Overall, the initialization method consisting of the Heu_Dominant and Heu_Time is effective.

6.2. Comparison of metaheuristics

For evaluating the performence of the proposed CDABC algorithm, we compare the Simulated Annealing algorithm (SA) [23], the Quantum Artificial Bee Colony algorithm (QABC) [27], the Ant Colony System algorithm (ACS) [29], the improved Artificial Bee Colony algorithm (IABC) [32], the discrete Artificial Bee Colony algorithm (DABC) [33], the Multi-picking-robot Discrete Artificial Bee Colony algorithm (MDABC) [35], and the Teaching-Learning-Based Optimizer algorithm (TLBO) [36]. The comparison algorithms parameters are also calibrated in the same way as for CDABC. The parameters of each comparison algorithm and a brief description are shown in Table 3.

All algorithms are used to solve 270 instances. In order to ensure a fair comparison, we adopt the same the maximum elapsed CPU time of $20 \times n \times m$ milliseconds as stopping criterion. All the experimental results are analyzed by the multifactor ANOVA method. For each metaheuristic, a total of $270 \times 10 = 2700$ *RPI* values are generated. The type of metaheuristics, the number of task (m) and the number of robots (n) are taken as factors. As shown in Table 4, the *p-value* is equal to zero,

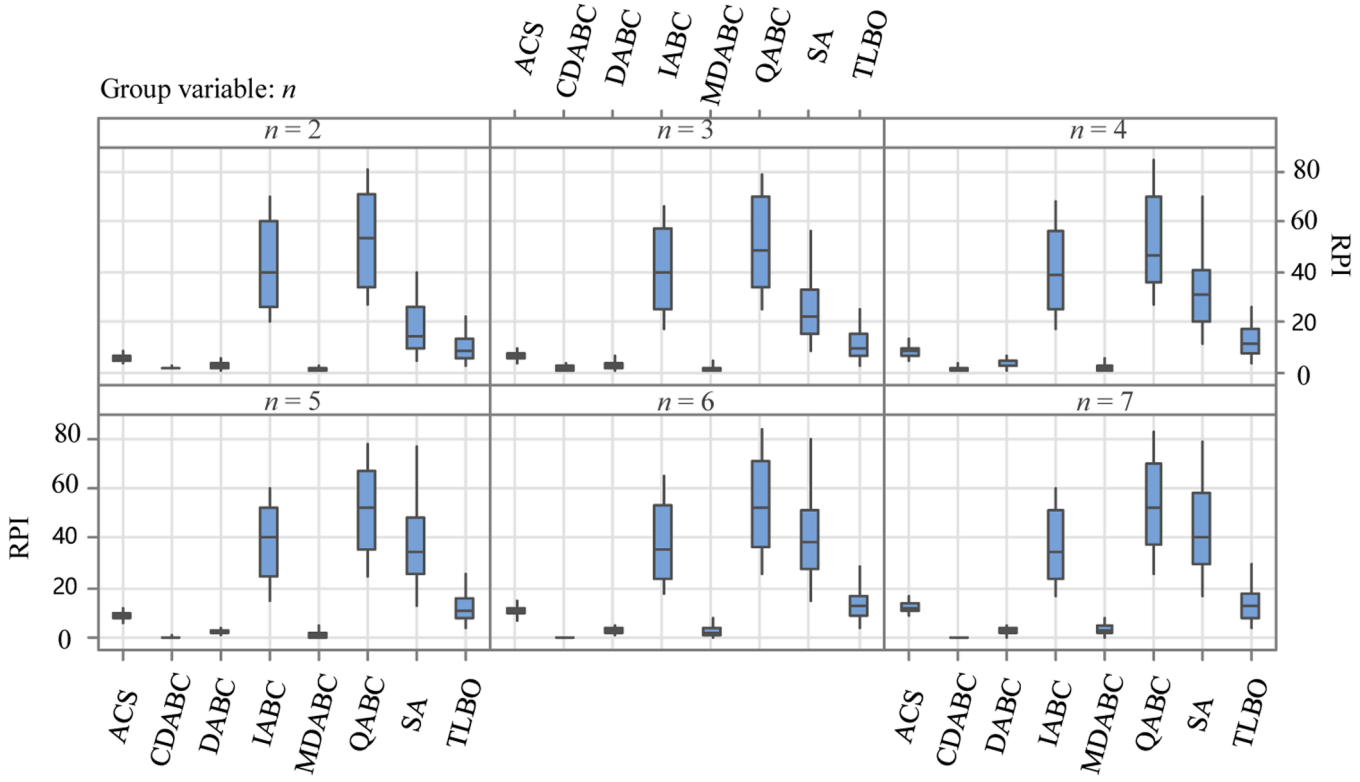


Fig. 11. The box plot of the compared algorithms grouped by the number of robots.

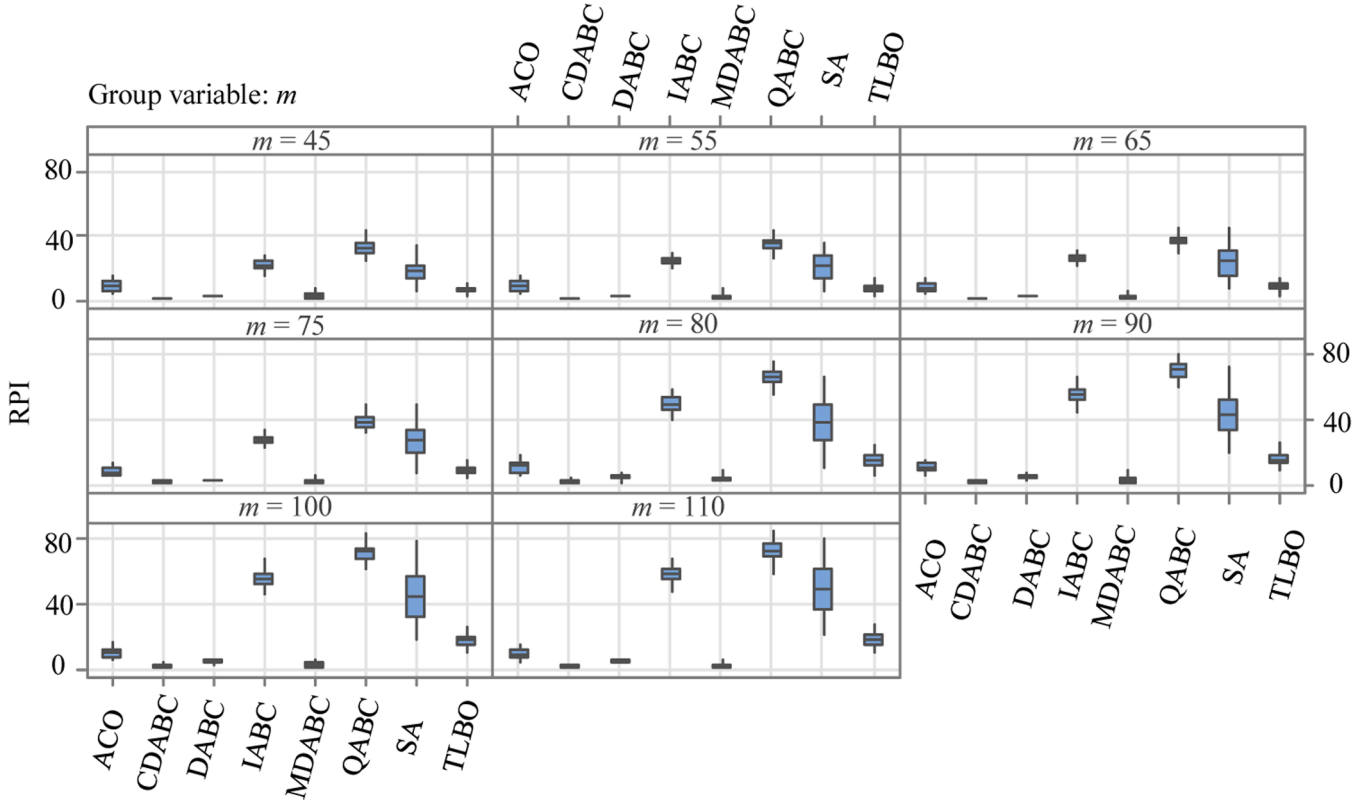


Fig. 12. The box plot of the compared algorithms grouped by the number of tasks.

indicating that the response variables (*RPI*) of the eight algorithms differed significantly at the 95 % confidence level. Fig. 6 is the means plot of the metaheuristics, which adopts the 95 % confidence level and Tukey HSD confidence interval.

It can be seen from Fig. 6 that *RPI* values generated by the eight algorithms are obviously different. CDABC achieves the best results, and then MDABC, DABC, ACS, TLBO, SA, IABC and QABC are ranked sequentially in the order from best to worst. Fig. 7 displays the means plots of interaction with 95 % Tukey HSD intervals. Fig. 7(a) represents the interaction plot between the compared algorithms and the number of robots, while Fig. 7(b) represents the interaction plot between the compared algorithms and the number of tasks. As is shown in Fig. 7(a), there are significant difference among the algorithms for different numbers of robots (n), and CDABC consistently outperforms others regardless of the number of robots. Compared with IABC, SA and QABC, CDABC, TLBO, MDABC, DABC and ACS have obvious advantages. When $n = 2$ and $n = 3$, CDABC and MDABC obtain similar results, and even when the number of robots is only 2, MDABC is slightly better than CDABC. However, with the increase of the number of robots, the advantage of CDABC become obvious. This is mainly because the cross-robot neighborhood search (MR-Insert and MR-Exchange) in the neighborhood strategy list and the exploitation of the critical robot in the collaborative onlooker bee stage reduce the maximum completion time. Fig. 7(b) shows the results grouped by the number of tasks (m). Compared with SA, IABC and QABC, the remaining algorithms obtain smoother *RPI* values and show a more stable performance. But CDABC is still the best one. As the number of tasks increases, especially for large-scale instances, each robot completes a large number of tasks. Their maximum completion time can be directly reduced by optimizing the task sequence for single robots, especially the critical robot. In CDABC, the utilization of SR-Insert, SR-Exchange, Criticalrobot-SR-Exchange and Criticalrobot-SR-Insert further enhances the algorithm's performance by providing more advantageous neighborhoods.

To further observe the differences between CDABC and other algorithms, we analyze the results by the Friedman-test. It is a non-parametric test which first assumes that all the compared algorithms are equivalent. If the original hypothesis is rejected, it suggests that there is a difference between the algorithms, otherwise, there is no difference between all algorithms. The detail analysis results are presented in Tables 5–7, where *CN* is the number of instances. It can be observed from these tables that CDABC is the best one. The *p-value* calculated from the test statistic is equal to 0 with a significance level of $\alpha = 0.05$, indicating significant differences between the compared algorithms. The results of the Friedman-test, grouped by the number of robots and the number of tasks, are visualized in Figs. 8 and 9. Fig. 8 shows that CDABC achieves best performance for all task sizes. This is due to the fact that it has both inner-robot and cross-robot exploitation, as well as the fact that CFS assists the individual to avoid falling into a local optimum. As can be seen from Fig. 9, the average rank of CDABC is the smallest, indicating CDABC outperforms all other algorithms. However, when $n = 2$ and $n = 3$, the same results are observed as in Fig. 7(a), and the difference between CDABC and MDABC is not significant. This is because MDABC has a greater random probability (50 %) of selecting critical robots when $n = 2$, that is to say, even though these algorithms select robots randomly, there is still a greater probability of selecting critical robots for exploitation, which somehow narrows the gap with CDABC. However, as the number of robots increases, the advantage of the critical robot becomes clear and CDABC obtains better results. Therefore, CDABC exhibits superior performance compared to the other algorithms with the 95 % confidence interval.

Furthermore, we assess the stability of CDABC by the box plots of the eight algorithms on *RPI* values. A more compact box plot indicates more stable performance, while a box plot near the bottom indicates better performance for the minimization problem. As shown in Fig. 10, CDABC is not only the box closest to the bottom, but also possesses the narrowest box. Further, the box plots grouped by the number of robots and the number of tasks are shown in Figs. 11 and 12, from which it can be seen that the proposed CDABC is the best one, and as the number of robots and tasks increases, the advantages of CDABC become increasingly clear.

7. Conclusions and future research

In this paper, we address the multi-robot task allocation and scheduling problem, focusing on weeding tasks undertaken by multiple agricultural robots in a smart farm. Our objective is to minimize the maximum completion time among multiple robots. To solve this problem, we develop two innovative heuristics, Heu_Dominant and Heu_Time, which help to provide high-quality initial solutions for CDABC. Another notable contribution of our work is the introduction of the collaborative following bee strategy (CFS) in CDABC. The introduced collaborative following bee strategy has a positive impact on the algorithm, which helps bees trapped in the local optimum to explore more promising regions. To obtain the better performance, we calibrate the parameters of CDABC by using DOE and ANOVA technology. Comparative experiments demonstrate the superior performance of our heuristics and collaborative following bee strategy, especially in minimizing the maximum completion time across multiple robots. Finally, the comprehensive experiments and statistical analysis confirm that the proposed CDABC are significantly better than the state-of-the-art methods in the literature.

In the future, we would consider more complex and practical constraints in MRTAS such as multi-objective optimization and fuzzy time, and design more effective heuristics and meta-heuristics based on the study of problem knowledge. In addition, problem-oriented strategies and energy-sensitive allocation strategies should be further studied.

CRediT authorship contribution statement

Hengwei Guo: Writing – original draft, Visualization, Methodology, Investigation. **Zhonghua Miao:** Writing – review & editing, Supervision, Software, Resources, Project administration. **JC Ji:** Writing – review & editing, Conceptualization. **Quanke Pan:** Writing – review & editing, Supervision, Software, Resources, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is supported by the Innovation Program of Shanghai Municipal Education Commission 2023ZKZD47, the Shanghai Agriculture Applied Technology Development Program 2020-02-08-00-09-F01466, the National Nature Science Foundation of China 51875331, 62273221 and 61973203, Program of Shanghai Academic/Technology Research Leader 21XD1401000.

References

[1] Walter, et al., Smart farming is key to developing sustainable agriculture, Proc. Natl. Acad. Sci. U. S. A. 114 (24) (2017) 6148–6150.
[2] Y. Wu, Y. Duan, Y. Wei, et al., Application of intelligent and unmanned equipment in aquaculture: a review, Comput. Electron. Agric. 199 (2022) 107201. Aug.

[3] X. Luo, L. Hu, J. He, et al., Key technologies and practice of unmanned farm in China, Trans. Chin. Soc. Agric. Eng. 40 (1) (2024) 1–16.
[4] I. Charania, X. Li, Smart farming: Agriculture's shift from a labor intensive to technology native industry, Internet Things 9 (2020) 100142. Mar.
[5] V.O. Abegunde, A. Obi, The role and perspective of climate smart agriculture in Africa: a scientific review, Sustainability 14 (4) (2022) 2317. Feb.
[6] X. Chen, P. Zhang, G. Du, et al., A distributed method for dynamic multi-robot task allocation problems with critical time constraints, Robot. Auton. Syst. 118 (2019) 31–46. Aug.
[7] F. da Silveira, F.H. Lermen, F.G. Amaral, An overview of agriculture 4.0 development: Systematic review of descriptions, technologies, barriers, advantages, and disadvantages, Comput. Electron. Agric. 189 (2021) 106405. Oct.
[8] M. Javaid, A. Haleem, R.P. Singh, et al., Enhancing smart farming through the applications of agriculture 4.0 technologies, Int. J. Intell. Netw. 3 (2022) 150–164. Jan.
[9] B. Zhou, R. Zhou, Y. Gan, F. Fang, Y. Mao, Multi-robot multi-station cooperative spot welding task allocation based on stepwise optimization: an industrial case study, Robot. Comput. Integr. Manuf. 73 (2022) 102197. Feb.
[10] S. Choudhury, J.K. Gupta, M.J. Kochenderfer, D. Sadigh, J. Bohg, Dynamic multi-robot task allocation under uncertainty and temporal constraints, Auton. Robots 46 (1) (2022) 231–247. Jan.
[11] J. Li, M. Song, L. Wang, et al., Hybrid artificial bee colony algorithm for a parallel batching distributed flow-shop problem with deteriorating jobs, IEEE Trans. Cybern. 50 (6) (2020) 2425–2439. Jun.
[12] W.Y. Szeto, Y. Wu, S.C. Ho, An artificial bee colony algorithm for the capacitated vehicle routing problem, Eur. J. Oper. Res. 215 (1) (2011) 126–135. Nov.
[13] S. Liu, P. Zhang, M. Zhu, ' Artificial bee colony algorithm based on local search, Control. Deci. 29 (1) (2014) 123–128.
[14] M.F. Tasgetiren, D. Yüksel, L. Gao, et al., A discrete artificial bee colony algorithm for the energy-efficient no-wait flowshop scheduling problem, Procedia Manuf. 39 (2019) 1223–1231. Jan.
[15] Q. Pan, L. Gao, L. Wang, et al., Effective heuristics and metaheuristics to minimize total flowtime for the distributed permutation flowshop problem, Expert Syst. Appl. 124 (2019) 309–324. Jun.
[16] B.P. Gerkey, M.J. Matari, A formal analysis and taxonomy of task allocation in multi-robot systems, Int. J. Robot. Res. 23 (9) (2004) 939–954.
[17] N. Atay, B. Bayazit, Mixed-integer Linear Programming Solution to Multi-Robot Task Allocation Problem, Washington Univ., USA, 2006. Ph.D. dissertation.
[18] B.P. Gerkey, M.J. Mataric, Sold!: auction methods for multirobot coordination, IEEE Trans Robot. Autom. 18 (5) (2002) 758–768.
[19] E.G. Jones, M.B. Dias, A. Stentz, Learning-enhanced market-based task allocation for oversubscribed domains. Proceedings of (IROS) IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2007, pp. 2308–2313.
[20] L. Luo, N. Chakraborty, K. Sycara, Competitive analysis of repeated greedy auction algorithm for online multi-robot task assignment, in: Proceeding of the Presented at IEEE International Conference on Robotics & Automation, 2012, pp. 4792–4799.
[21] F. Zitouni, S. Harous, R. Maamri, A distributed approach to the multi-robot task allocation problem using the consensus-based bundle algorithm and ant colony system, IEEE Access 8 (2020) 27479–27494.
[22] D. Juedes, F. Drews, L. Welch, et al., Heuristic resource allocation algorithms for maximizing allowable workload in dynamic, distributed real-time systems, in: Proceedings of the Presented at 18th International Parallel and Distributed Processing Symposium, 2004, pp. 1631–1638. Apr.
[23] S. Bi, B. Shi, X. Liu, et al., MTSP's improved simulated annealing algorithm and its solution. China Management Information 25, 2022, pp. 158–161.
[24] E.G. Jones, M.B. Dias, A. Stentz, Time-extended multi-robot coordination for domains with intra-path constraints, Auton. Robot. 30 (1) (2011) 41–56. Jan.
[25] D. Zhang, G. Xie, J. Yu, et al., Adaptive task assignment for multiple mobile robots via swarm intelligence approach, Robot. Auton. Syst. 55 (7) (2007) 572–588. Jul.
[26] F. Zitouni, R. Maamri, FA-SETPOWER-MRTA: A solution for solving the multi-robot task allocation problem, in: Proceedings of the Presented in IFIP International Conference, Oran, Algeria, 2018, pp. 317–328.
[27] F. Zitouni, R. Maamri, S. Harous, FA–QABC–MRTA: a solution for solving the multi-robot task allocation problem, Intell. Serv. Robot. 12 (4) (2019) 407–418. Oct.
[28] Y. Ding, Y. He, J. Jiang, Multi-robot cooperation method based on the ant algorithm, in: Proceedings of the 2003 IEEE Swarm Intelligence Symposium, USA, 2003, pp. 14–18. . INApr.
[29] R. Necula, M. Raschip, M. Breaban, Balancing the subtours for multiple TSP approached with ACS: clustering-based approaches Vs. MinMax formulation, Adv. Intell. Syst. Comput. 674 (2018) 210–223.
[30] F. Xue, T. Dong, S. You, et al., A hybrid many-objective competitive swarm optimization algorithm for large-scale multirobot task allocation problem, Int. J. Mach. Learn. Cybern. 12 (4) (2021) 943–957. Apr.
[31] J. Wang, Y. Gu, X. Li, Multi-robot task allocation based on ant colony algorithm, J. Comput. 7 (9) (2012) 2160–2167. Sep.
[32] X. Zhang, B. Zhu, Solution of robot task assignment based on improved artificial bee colony algorithm, Int. Comput. Appl. 12 (8) (2022) 157–160.
[33] W. Zou, Q. Pan, T. Meng, et al., An effective discrete artificial bee colony algorithm for multi-AGVs dispatching problem in a matrix manufacturing workshop, Expert Syst. Appl. 161 (2020) 113675. Dec.
[34] M. Badreldin, A. Hussein, A. Khamis, A comparative study between optimization and market-based approaches to multi-robot task allocation, Adv. Artif. Intell. 2013 (2013) 1–11. Nov.
[35] L. Dai, Q. Pan, Z. Miao, et al., Multi-objective multi-picking-robot task allocation: mathematical model and discrete artificial bee colony algorithm, IEEE T. Intell. Transp. (2023) in press.
[36] N. Kang, Z. Miao, Q. Pan, et al., A multi-objective teaching-learning-based optimizer for a multi-weeding-robot task assignment problem, Tsinghua Sci. Technol. 29 (2024) 1–17. Oct.
[37] I.F.A. Vis, Survey of research in the design and control of automated guided vehicle systems, Eur. J. Oper. Res. 170 (3) (2006) 677–709.
[38] T. Utstumo, F. Urdal, A. Brevik, et al., Robotic in-row weed control in vegetables, Comput. Electron. Agric. 154 (2018) 36–45. Nov.
[39] J. Huang, Q. Pan, Z. Miao, et al., Effective constructive heuristics and discrete bee colony optimization for distributed flowshop with setup times, Eng. Appl. Artif. Intell. 97 (2021) 104016. Jan.
[40] H. Guo, H. Sang, B. Zhang, et al., An effective metaheuristic with a differential flight strategy for the distributed permutation flowshop scheduling problem with sequence-dependent setup times, Knowl. Based Syst. 242 (2022) 108328.
[41] J. Li, Q. Pan, K. Mao, et al., Solving the steelmaking casting problem using an effective fruit fly optimisation algorithm, Knowl. Based Syst 72 (2014) 28–36. Dec.
[42] L. Meng, C. Zhang, Y. Ren, et al., Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem, Comput. Ind. Eng. 142 (2020) 106347. Apr.
[43] L. Meng, K. Gao, Y. Ren, et al., Novel MILP and CP models for distributed hybrid flowshop scheduling problem with sequence-dependent setup times, Swarm Evol. Comput. 71 (2022) 101058. Jun.
[44] Z. Shao, W. Shao, D. Pi, Effective constructive heuristic and iterated greedy algorithm for distributed mixed blocking permutation flow-shop scheduling problem, Knowl. Based Syst 221 (2021) 106959. Jun.